Smart Expense Tracker

with Goal-Based Spending Analysis

AI/ML Fundamentals Course — Project Report

Student	Shaurya Anurag
GitHub	github.com/Shaurya-Anurag/smart-expense-tracker
Reg. No.	25BCE10395
Tech Stack	Python 3.13 | Streamlit | Matplotlib
Date	March 2026

1. Introduction

Managing personal finances is a common challenge for students and working professionals alike. Most existing apps are either too complex or require subscriptions. This project delivers a lightweight, locally-running expense tracker that uses Artificial Intelligence concepts to automatically understand and classify spending — making budgeting both smarter and easier.

The application is built entirely in Python with no external database, no paid APIs, and no heavy ML frameworks. It runs on any laptop in under a minute of setup.

1.1 Project Objectives

- Accept natural-language expense input such as "250 swiggy" and automatically extract amount, description, and category.
- Classify expenses into 6 categories using a rule-based AI classifier.
- Provide statistical analysis: totals, averages, and category breakdowns.
- Allow users to set financial goals and track progress with intelligent alerts.
- Present all data through a clean, dark-themed Streamlit web interface.

1.2 Academic Alignment

AI/ML Concepts Demonstrated

Classification | Statistical Analysis | Rule-Based Decision System | Bag-of-Words intuition

2. System Architecture

The project follows a modular, single-responsibility architecture. Each Python file owns exactly one concern, making the code easy to read, test, and extend.

File	Module	Responsibility
app.py	UI Layer	Streamlit interface — all 4 pages, charts, CSS
classifier.py	AI Engine	Parses raw input; keyword-based category classifier
expense_manager.py	Data Layer	CSV read/write for all expense records
goal_manager.py	Goal Layer	JSON read/write for financial goals
analyzer.py	Analytics	Statistics engine + rule-based alert system

2.1 Data Flow

The system follows a clean pipeline for every expense entry:

> **Pipeline**
> User types input → classifier.py parses & classifies → expense_manager.py saves to CSV → analyzer.py computes stats → app.py renders charts & alerts

2.2 Storage Design

- **expenses.csv** — append-only flat file. One row per expense: date, amount, description, category.
- **goals.json** — JSON array of goal objects. Supports multiple goals; most recent is treated as active.
- **— the data folder is created automatically on first run.**

3. Expense Classifier (AI Core)

The classifier is the AI heart of the project. It demonstrates classification — one of the most fundamental AI/ML tasks — using an interpretable, rule-based approach that requires zero training data.

3.1 How Classification Works

When a user types "250 swiggy biryani", the classifier performs two steps:

Step	Operation	Example
1	Regex extraction	Finds first number: amount = 250.0
2	Keyword scoring	"swiggy" matches Food keywords → score[Food]++
3	argmax	Category with highest score wins → Food

3.2 Category Keyword Map

Category	Sample Keywords
Food	swiggy, zomato, restaurant, cafe, biryani, grocery, milk, eggs, canteen...
Transport	uber, ola, auto, metro, train, bus, petrol, rapido, toll, flight...
Entertainment	netflix, spotify, hotstar, movie, cinema, game, concert, club...
Essentials	rent, electricity, wifi, insurance, emi, medicine, hospital, school fee...
Shopping	amazon, flipkart, myntra, clothes, shoes, gadget, salon, cosmetics...
Other	Fallback — assigned when no keywords match

3.3 Connection to Naive Bayes

Academic Note

The keyword scoring approach mirrors the Bag-of-Words representation used in Naive Bayes text classification. Instead of learned probabilities P(word|class), we use binary presence scores. The argmax selection is equivalent to MAP classification. This makes the system explainable: you can always trace exactly why a category was assigned.

4. Statistical Analysis Engine

The analyzer.py module computes descriptive statistics over the expense dataset and drives all charts in the UI.

4.1 Metrics Computed

Metric	Formula	Purpose
Total Spending	Σ amount_i	Overall financial footprint
Average Spending	Σ amount_i / n	Typical transaction size
Category Breakdown	Σ amount_i per category	Identifies spending habits
Highest Category	argmax(breakdown)	Pinpoints biggest cost driver
Daily Trend	Σ amount_i per date	Temporal spending pattern
Monthly Spending	Filter by current month	Used for goal tracking

4.2 Visualisations

- **Pie Chart** — category breakdown as proportions (donut style, matplotlib).
- **Horizontal Bar Chart** — absolute amounts per category for easy comparison.
- **Line Chart** — daily spending trend over time with fill-under area.

5. Goal System & Intelligent Alerts

The goal system turns the tracker into a proactive financial assistant. It demonstrates a rule-based decision system — a core concept in classical AI.

5.1 Goal Setup

The user provides three inputs: Goal Name, Target Amount (₹), and Timeframe (months). The system derives:

Formula

Monthly Saving Required = Target Amount ÷ Months Example: ₹60,000 goal in 6 months → must save ₹10,000 per month

5.2 Alert Rules (Rule-Based AI Agent)

Rule	Condition	Alert Fired
1	Spent > pro-rated monthly budget	□ Overspending warning — risk of missing goal
2	Spent ≤ pro-rated budget	✅ On track confirmation + estimated surplus
3	Any category > 50% of monthly saving target	□ Category overspend warning

Pro-rated budget = (days elapsed / days in month) × (target − monthly saving required). This ensures the alert is fair — spending ₹5,000 on day 1 of the month is evaluated differently than on day 28.

6. User Interface

The UI is built with Streamlit and uses custom CSS for a dark, modern aesthetic inspired by developer tools. Navigation is handled through a sidebar with 4 pages.

6.1 Pages

Page	Features
Add Expense	Smart input box with live preview card showing parsed amount, description, and category pill before saving
Set Goal	Form for name, target amount, and months. Displays all saved goals with monthly saving requirement and delete button
Analysis	4 KPI cards, pie chart, bar chart, daily trend line, and full goal analysis panel with alerts
All Expenses	Filterable, sortable table of every expense with category pills and clear-all option

6.2 Design Decisions

- **Dark theme (#0f0f13 background)** — reduces eye strain during extended use.
- **Live preview** — user sees classification result before committing, building trust in the AI.
- **Color-coded category pills** — instant visual recognition of spending type.
- **Sidebar quick stats** — total spent and current month always visible regardless of active page.
- **Balloon animation** — positive reinforcement when an expense is successfully saved.

7. Example Inputs & Outputs

7.1 Expense Classification Examples

Raw Input	Amount	Description	Category
250 swiggy	₹250	Swiggy	□ Food
1200 uber airport	₹1,200	Uber Airport	□ Transport
499 netflix	₹499	Netflix	□ Entertainment
15000 rent june	₹15,000	Rent June	□ Essentials
3500 amazon laptop bag	₹3,500	Amazon Laptop Bag	□ Shopping
abc (invalid)	—	—	❌Rejected

7.2 Goal Analysis Example

Goal Setup

Goal: New Laptop | Target: ₹60,000 | Timeframe: 6 months | Monthly Saving Required: ₹10,000

Rule 1 Fires

Monthly spending: ₹1,949 | Pro-rated limit: ₹45,161 | Status: ✅On Track | Estimated surplus: ₹43,212

Rule 3 Example

If Transport spending = ₹6,000 > ₹5,000 (50% of target) → □ Category overspend warning fires

8. Installation & Running

8.1 Prerequisites

- Python 3.10 or higher
- pip or conda package manager
- VS Code (recommended) or any terminal

8.2 Setup Steps

Step	Command / Action
1	Download all 5 .py files + requirements.txt into one folder
2	pip install streamlit matplotlib
3	streamlit run app.py
4	Open browser at http://localhost:8501

8.3 GitHub Repository

The complete source code is publicly available at:

Repository
https://github.com/Shaurya-Anurag/smart-expense-tracker

9. Limitations & Future Work

9.1 Current Limitations

- The classifier uses exact keyword matching — misspellings like "swiggyy" will not match.
- No income tracking — goal alerts assume spending data alone without an income baseline.
- Single-user — the CSV/JSON storage is not designed for concurrent multi-user access.
- No data encryption — expense data is stored as plain text locally.

9.2 Potential Improvements

- **Naive Bayes classifier** — train on a labelled expense dataset for probabilistic classification.
- **Fuzzy matching** — handle typos using Levenshtein distance.
- **Income tracking** — add monthly income input for true savings rate calculation.
- **Export to PDF** — generate monthly spending reports automatically.
- **SMS/email alerts** — push notifications when spending thresholds are crossed.

10. Conclusion

The Smart Expense Tracker successfully demonstrates three core AI/ML concepts in a single, practical application:

- **Classification** — rule-based keyword scoring assigns expenses to categories without any training data.
- **Statistical Analysis** — descriptive statistics (mean, sum, breakdown) are computed and visualised in real time.
- **Rule-Based AI Agent** — three explicit decision rules generate contextual alerts that guide user behaviour.

The project is intentionally simple — two Python packages, two data files, five modules — yet it is fully functional, visually polished, and academically rigorous. It demonstrates that powerful, intelligent applications do not require large language models or GPU clusters; thoughtful engineering of classical AI techniques is often sufficient.

> **Key Takeaway**
> Good AI is not always the most complex AI. A well-designed rule-based system with clear logic, proper validation, and a great UI can outperform a black-box model in real-world usability and trust.

— End of Report —